

汇编语言大作业 2 实验报告

小组成员：吴竞宸、黄奕皓、张晨轩

2026/05/26

1 概述

本次实验采用小组合作的方式完成，开发环境为 Linux，成员通过微信群交流开发进度，使用 Github 共同管理代码。

Github 仓库网址为：<https://github.com/Dandelom/UCAS-ASM-PRJ2>

实验要求实现 4096×4096 单精度浮点的矩阵转置程序，在实现简单转置的基础上，我们采用了 4 种优化方法，分别为：

1. 分块转置 (64×64)
2. 4×4 SIMD 转置
3. 8×8 AVX 转置
4. AVX 转置 + Prefetch

其中，分块转置以及最先实现的简单转置使用 x86-32 指令集，而后三种优化方式使用 x86-64 指令集完成，分别由 main32.c 和 main64.c 控制。

建议的编译指令：

```
gcc -m32 main32.c simply_transpose.s blockwise_transpose.c diagonal_transpose.s off_diagonal_transpose.s -o ../bin/transpose32
```

```
gcc main64.c simd_block_transpose.s simd_transpose.c avx_block_transpose.s avx_transpose.c avx_prefetch.c avx_dual_prefetch_transpose.s avx_prefetch_transpose.s -o ../bin/transpose64
```

src 文件夹下还有 generate.c 用于生成待转置的 4096×4096 单精度浮点矩阵，采用二进制文件形式存储；check.c 用于检查转置是否正确。测试数据和输出数据（.bin 文件）存放在 tests 文件夹下。

建议的编译指令：

```
gcc generate.c -o ../bin/generate
```

```
gcc check.c -o ../bin/check
```

2 实验结果

2.1 实验数据

程序运行环境为 Windows 11 下 VMware Workstation 内的 Ubuntu 24.04.3 虚拟机。为虚拟机分配的内存为 6 GB，处理器内核总数为 4。物理机的 CPU 型号为 Intel(R) Core(TM) i9-14900HX (2.20 GHz)。

为简单转置及各加速方法采用 5 次运行取平均值的方法，原始数据记录如下。

单位：ms

	简单转置	分块转置	SIMD	AVX	AVX + Prefetch
1	72.81	38.35	53.50	22.69	11.69
2	70.73	37.25	50.87	24.38	11.80
3	76.10	38.19	48.18	24.29	11.46
4	66.00	38.85	48.86	23.52	14.11
5	69.96	39.56	49.35	27.69	13.41

表 1: 原始数据

计算得到最终算法时间与加速比如下：

算法	运行时间 (ms)	加速比
简单转置	71.12	1.00
分块转置	38.44	1.85
SIMD	50.15	1.42
AVX	24.51	2.90
AVX + Prefetch	12.49	5.69

表 2: 算法时间与加速比

2.2 算法分析

普通转置在处理行和列时，由于大矩阵（ 4096×4096 ）的单行数据量远超 L1/L2 缓存容量，按列读取或写入时会产生巨大的跨步访问（Stride Visit），导致严重的 Cache Line 缺失（Cache Miss）。分块转置将大矩阵切分为局部的小块，使得小块的数据可以完全容纳在高速缓存（L1 Cache）中，极大地提高了时间局部性和空间局部性，加速比达到了 1.85。

从数据上看，4x4 SIMD 的加速比低于普通分块转置。分析代码，可能的原因是频繁的 memcpy 带来的数据吞吐损耗抵消了 SIMD 节省下来的计算时间。

而 8x8 AVX 转置在分块的基础上利用了 SIMD（单指令多数据）寄存器。AVX 拥有 256 位的 %ymm

寄存器,单条指令可以同时处理 8 个单精度浮点数。它的核心优势是用硬件寄存器内的洗牌指令(Shuffle/Unpack/Permute)替代了传统的标量交换循环,消除了分支预测失败的开销,并最大化了指令级并行度(ILP)。加速比达到 2.90。

AVX + Prefetch 转置是性能发生质飞跃的一步。即使数据在 L1 缓存里处理得再快,当跳到下一个物理上不连续的内存块时,CPU 仍然需要停下来等待总线从主存中加载数据。通过在 C 语言端计算下一轮循环的地址并传入汇编,利用 prefetcht0 指令在当前块执行洗牌计算的同时,异步将下一个块的数据提前拉入 L1 缓存。这就完美掩盖了内存访问延迟(Memory Latency),使 CPU 核心几乎不用处于空转(Stall)状态。这使得加速比来到了最高的 5.69。